PRACTICAL – 1

Implementation and Time Analysis of Sorting Algorithms

Objective:

The objective of this practical is to implement and compare the performance of ***Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort***. We will analyze the time complexity and execution time of each algorithm to understand their efficiency and identify the most suitable algorithm for different applications.

1. Bubble Sort
Algorithm:

1. Start

2.For i= 0 to n-2

3. For j=0 to n-i-2

4. IfA[j]>A[j+1]

5. SwapA[j]and A[j + 1]

6. End If

7. End For

8. End For

9. Stop

Code:

```
import time

# Bubble Sort Function
def bubble_sort(arr, n):
    for i in range(n - 1):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swapping occurs, array is already sorted
        if not swapped:
            break


# Take input from the user
n = int(input("Enter the number of elements: "))
```

```
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Measure execution time
start_time = time.perf_counter()

bubble_sort(arr, n)

end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f}
seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Users\penta\AppData\Loca
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60960' '--
74
657
Sorted array: 1 3 3 4 28 74 94 657 893

Time Complexity:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)
Enrollment No: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n)*	*The array is already sorted. With the optimized version (using a swapped flag), only one pass is required.*
Average Case	*$O(n^2)$*	*The array is in random order, requiring multiple comparisons and swaps.*
Worst Case	*$O(n^2)$*	*The array is sorted in reverse order, requiring the maximum number of comparisons and swaps.*

Space Complexity:

Space Complexity	*Description*
O(1)	*Uses only a temporary variable for swapping.*

2. Selection Sort

Algorithm:

1. Start
2. For i = 0 to n - 2
3. min = i
4. For j = i + 1 to n - 1
5. If A[j] < A[min]
6. min = j
7. End If
8. End For
9. Swap A[i] and A[min]
10. End For
11. Stop

Code:

```
import time

# Selection Sort Function
def selection_sort(arr):
    n = len(arr)
```

```
    for i in range(n):
        min_index = i

        # Find the minimum element in the unsorted part
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum element with the first unsorted element
        arr[i], arr[min_index] = arr[min_index], arr[i]


# Take input from the user
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Measure execution time
start_time = time.perf_counter()

selection_sort(arr)

end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\deb
25
Sorted array: 25 35 78 98

Time Complexity:
Best Case: O(n^2)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*$O(n^2)$*	*Even if the array is already sorted, the algorithm scans the remaining unsorted elements to find the minimum in every pass.*
Average Case	*$O(n^2)$*	*The array is in random order, requiring the same number of comparisons in each pass.*
Worst Case	*$O(n^2)$*	*The array is in reverse order, but the algorithm still performs the same number of comparisons and selects the minimum element in each pass.*

Space Complexity:

Space Complexity	*Description*
O(1)	*Sorts in place using constant extra memory.*

3. Insertion Sort

Algorithm:

1. Start
2. For i = 1 to n - 1
3. key = A[i]
4. j = i - 1
5. While j >= 0 AND A[j] > key
6. A[j + 1] = A[j]
7. j = j - 1
8. End While
9. A[j + 1] = key
10. End For
11. Stop

Code:

```
import time

# Insertion Sort Function
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1
        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
insertion_sort(arr)
end_time = time.perf_counter()
# Display the sorted array
print("\nSorted Array:", arr)
```

```
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Use
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
62
12
14
Sorted array: 12 14 45 62 89
Time Complexity:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)
Enrollment No: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n)*	The array is already sorted, so each element is compared only once without any shifting.
Average Case	*O(n²)*	The array is randomly ordered, requiring comparisons and shifts for inserting elements into their correct positions.
Worst Case	*O(n²)*	The array is sorted in reverse order, requiring the maximum number of comparisons and shifts for every element.

Space Complexity:

Space Complexity	*Description*
O(1)	*Uses one extra variable for inserting elements.*

4. Merge Sort :

Algorithm:

1. If left < right
2. mid = (left + right) / 2
3. MergeSort(A, left, mid)
4. Merge Sort(A, mid + 1, right)
5. Merge(A, left, mid, right)
6. End If

Code:

```
import time
# Merge Sort Function
def merge_sort(arr):
    if len(arr) > 1:
        # Find the middle of the array
        mid = len(arr) // 2
        # Divide the array into two halves
        left = arr[:mid]
        right = arr[mid:]
        # Recursively sort both halves
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        # Merge the sorted halves
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        # Copy remaining elements of left half
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
```

```
        # Copy remaining elements of right half
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
merge_sort(arr)
end_time = time.perf_counter()
# Display the sorted array
print("\nSorted Array:", arr)
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\Users\pent
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
37
Sorted array: 9 35 37 74 89

Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n log n)
Space Complexity: O(n)

Enrollment Number:92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n log n)*	*The array is recursively divided and merged efficiently regardless of its initial order.*
Average Case	*O(n log n)*	*Randomly ordered elements are divided into subarrays and merged in sorted order.*
Worst Case	*O(n log n)*	*Even in the worst arrangement, the algorithm performs the same divide-and-conquer operations.*

Space Complexity:

Space Complexity	*Description*
O(n)	*Uses an additional array during merging.*

5. Quick Sort:

Algorithm:

```
if low < high:
    pivot = Partition(arr, low, high)
    QuickSort(arr, low, pivot - 1)
    QuickSort(arr, pivot + 1, high)
return arr
```

Code:

```
import time
# Partition Function
def partition(arr, low, high):
    pivot = arr[high]      # Choose last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
# Quick Sort Function
def quick_sort(arr, low, high):
    if low < high:
```

```
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
# Measure execution time
start_time = time.perf_counter()
quick_sort(arr, 0, n - 1)
end_time = time.perf_counter()

# Display the sorted array
print("\nSorted Array:", arr)

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))
#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-1-main>  & 'C:\U
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugp
99
Sorted array: 46 87 94 99

Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n^2)
Space Complexity: O(log n) average

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n log n)*	*The pivot divides the array into two nearly equal halves at every recursive step.*
Average Case	*O(n log n)*	*On average, the pivot creates reasonably balanced partitions, resulting in efficient sorting.*
Worst Case	*O(n²)*	*The pivot is always the smallest or largest element, creating highly unbalanced partitions (e.g., an already sorted array with a poor pivot choice).*

Space Complexity:

Space Complexity	*Description*
O(log n) (Average)	Uses recursion stack during sorting.
O(n) (Worst Case)	Occurs when partitions are highly unbalanced.

PRACTICAL – 2

Implementation and Time Analysis of Linear and Binary Search Algorithm

Objective:

The objective of this practical is to implement ***Linear Search*** and ***Binary Search*** algorithms and compare their performance based on ***time complexity***, ***space complexity***, and ***execution time***. This helps in understanding the efficiency of both searching techniques and selecting the appropriate algorithm for different applications.

1.Linear Search

Algorithm:

START

INPUT A
INPUT X

FOR i ← 0 TO len(A) - 1
IF A[i] == X THEN

PRINT "Element found at index", i
EXIT
END IF
END FOR

PRINT "Element not found"

STOP

PythonCode:

```
import time
# Linear Search Function
def linear_search(arr, n, key):
    for i in range(n):
        if arr[i] == key:
            return i
    return -1
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search: "))
# Measure execution time
start_time = time.perf_counter()
result = linear_search(arr, n, key)
```

```
end_time = time.perf_counter()
# Display result
if result != -1:
    print("\nElement found at position:", result + 1)
else:
    print("\nElement not found")
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))

#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-2-main>  & 'C:\Users\pent
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
earsearch.py'
Enter the number of elements: 4
Enter 4 elements:
66
90
34
67
Enter the element to search: 90
Element found at position: 2

Time Complexity:
Best Case: O(1)
Average Case: O(n)
Worst Case: O(n)
Space Complexity: O(1)

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(1)*	*Element is at the first position.*
Average Case	*O(n)*	*Element is in the middle or random position.*
Worst case	*O(n)*	*Element is at the last position or absent.*

Space Complexity:

Space Complexity	***Description***
O(1)	***Uses only a few variables; no extra memory is required.***

2.Binary Search

Algorithm:

START

INPUT sorted list A

INPUT X

low ← 0

high ← len(A)-1

WHILE low <=high

mid ← (low+high)//2

IF A[mid]==XTHEN

PRINT"Elementfoundatindex",mid

EXIT

ELSEIFA[mid]<XTHEN

low←mid+1

ELSE

high←mid-1

END IF

ENDWHILE

PRINT "Element not found"

STOP

Python Code

```
import time
# Binary Search Function
def binary_search(arr, n, key):
    low = 0
    high = n - 1
    while low <= high:
```

```
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search: "))
# Sort the array
arr.sort()
# Measure execution time
start_time = time.perf_counter()
result = binary_search(arr, n, key)
end_time = time.perf_counter()
# Display sorted array
print("\nSorted Array:", arr)
# Display result
if result != -1:
    print("Element found at position:", result + 1)
else:
    print("Element not found")

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))

#Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-2-main>  & 'C:\Users\pent
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
arysearch.py'
Enter the number of elements: 5
Enter 5 elements in sorted order:
45
78
93
67
22
Enter the element to search: 93
Element found at position: 3

Time Complexity:
Best Case: O(1)
Average Case: O(log n)
Worst Case: O(log n)
Space Complexity: O(1)

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(1)*	*Element is found at the middle.*
Average Case	*O(log n)*	*Search space is halved repeatedly.*
Worst case	*O(log n)*	*Maximum divisions before finding or rejecting the element.*

Space Complexity:

Space Complexity	*Description*
O(1)	*Uses only a few variables (low, high, mid); no extra memory is required.*

PRACTICAL – 3

Implementation of Max-Heap Sort Algorithm

Objective:

*The objective of this practical is to **implement the Max Heap Sort algorithm** using Python. The algorithm arranges the elements into a **Max Heap** and repeatedly extracts the **maximum element** to sort the array in ascending order.*

Max Heap Sort

Algorithm:

Start
For i = n/2 - 1 to 0
Apply MaxHeapify(A, n, i)
End For
For i = n - 1 to 1
Swap A[0] and A[i]
Apply MaxHeapify(A, i, 0)
End For
Stop

Heap Sort

Algorithm:

Start
Build a Max Heap from the given array A.
For i = n - 1 to 1
Swap A[0] and A[i]
Apply MaxHeapify(A, i, 0)
End For
Stop

Python Code:

```
# Max Heapify Function
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    # Check left child
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check right child
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Heapify the affected subtree
        heapify(arr, n, largest)

# Heap Sort Function
def heap_sort(arr):
    n = len(arr)

    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]

        # Heapify the reduced heap
        heapify(arr, i, 0)
```

```
# Take input from the user
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Display original array
print("\nOriginal Array:", arr)

# Perform Heap Sort
heap_sort(arr)

# Display sorted array
print("Sorted Array:", arr)

# Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-3-main>  & 'C
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\deb
HeapSort.py'
Enter the number of elements: 4
Enter 4 elements:
34
45
67
78
Sorted array: 34 45 67 78

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n log n)*	*The heap construction and extraction operations take O(n log n).*
Average Case	*O(n log n)*	*Each extraction requires heapification, giving O(n log n).*
Worst Case	*O(n log n)*	*Heap Sort still performs in O(n log n).*

Space Complexity:

Space Complexity	*Description*
O(log n)	*The recursive heapify() function uses the recursion stack.*

PRACTICAL – 4

Implementation and Time analysis of factorial program using iterative and recursive method

Aim:
To implement and analyze the time complexity of a factorial program using both iterative and recursive methods, and compare their performance.

Objective:
The objective of this experiment is to implement a factorial program using both iterative and recursive methods and analyze their performance. The experiment aims to understand the working of iteration and recursion, determine the time and space complexity of both approaches, and compare their efficiency in calculating the factorial of a given number.

1.Iterative Method

Algorithm:

- Start.
- Read n.
- Set fact = 1.
- Repeat from i = 1 to n:
- fact = fact * i
- Display fact.
- Stop.

Code:

```
# Iterative Factorial Function
def factorial_iterative(n):
    fact = 1

    for i in range(1, n + 1):
        fact = fact * i
```

```
    return fact


# Take input
n = int(input("Enter a number: "))

# Calculate factorial
result = factorial_iterative(n)

# Display result
print("Factorial using Iterative Method:", result)

# Enter your EN.No
print("EN.No:92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA-Practical-4-main>  & 'C:\Users\penta\AppData\Loca
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57519' '--
torial.py'
Enter a number: 34

Factorial using Iterative Method: 295232799039604140847618609643520000000
Factorial using Recursive Method: 295232799039604140847618609643520000000

Time Analysis:
Iterative Method: O(n)
Recursive Method: O(n)

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n)*	*Performs n iterations.*
Average Case	*O(n)*	*Performs n iterations.*
Worst case	*O(n)*	*Performs n iterations.*

Space Complexity:

Space Complexity	*Description*
O(1)	*Uses constant extra space.*

2.Recursive Method

Algorithm:

- Start.
- Read n.
- If n = 0 or n = 1, return 1.
- Otherwise, return n × factorial(n-1).
- Display the result.
- Stop.

Code:

```
# Recursive Factorial Function
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)


# Take input
n = int(input("Enter a number: "))

# Calculate factorial
result = factorial_recursive(n)

# Display result
print("Factorial using Recursive Method:", result)

# Enter your EN.No
print("EN.No:92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA-Practical-4-main>  & 'C:\Users\penta\AppData\Loca
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57519' '--
torial.py'
Enter a number: 34

Factorial using Iterative Method: 295232799039604140847618609643520000000
Factorial using Recursive Method: 295232799039604140847618609643520000000

Time Analysis:
Iterative Method: O(n)
Recursive Method: O(n)

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n)*	*Makes n recursive calls.*
Average Case	*O(n)*	*Makes n recursive calls*
Worst case	*O(n)*	*Makes n recursive calls*

Space Complexity:

Space Complexity	*Description*
O(n)	*Uses recursion stack for n function calls.*

PRACTICAL – 5

Implementation of a knapsack problem using dynamic programming.

Aim:
To implement the 0/1 Knapsack Problem using Dynamic Programming and determine the maximum value that can be obtained by selecting items within a given knapsack capacity.

Objective:
The objective of this experiment is to understand and implement the 0/1 Knapsack Problem using the Dynamic Programming technique. It aims to determine the optimal selection of items based on their weights and values while ensuring that the total weight does not exceed the given capacity. The experiment also helps in understanding the concept of optimal substructure and overlapping subproblems used in dynamic programming.

Algorithm:
1. **Start.**
2. Readthe number of items n.
3. Readthe **weight** and **value** of each item.
4. Readthe maximum capacity W of the knapsack.
5. Createa DP table dp[n+1][W+1].
6. Initialize the first row and first column of the table to 0.
7. For each item i from 1 to n:
   - o For each capacity w from 1 to W:
   - o If the weight of the current item is less than or equal to w, calculate:
   - o dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w])
   - o Otherwise: dp[i][w] = dp[i-1][w]
8. The value stored in dp[n][W] represents the **maximum value** that can be obtained.
9. Display the maximum value.
10. **Stop.**

Code:

```
def knapsack(weights, profits, capacity, n):
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):

            if weights[i - 1] <= w:
                dp[i][w] = max(
                    profits[i - 1] + dp[i - 1][w - weights[i - 1]],
```

```
                dp[i - 1][w]
            )
        else:
            dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]


n = int(input("Enter the number of items: "))

weights = []
profits = []

print("Enter weights:")
for i in range(n):
    weights.append(int(input()))

print("Enter profits:")
for i in range(n):
    profits.append(int(input()))

capacity = int(input("Enter knapsack capacity: "))

max_profit = knapsack(weights, profits, capacity, n)

print("\nMaximum Profit:", max_profit)

print("\nEnrollment Number: 92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\penta\AppData\Loc
py-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '65060' '-
Enter the number of items: 5
Enter weights:
5
3
6
4
2
Enter profits:
2
4
6
3
2
Enter knapsack capacity: 4

Maximum Profit: 4

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(nW)*	*DP table is filled for all items and capacities.*
Average Case	*O(nW)*	*Each item is checked for every capacity.*
Worst case	*O(nW)*	*All n × W table entries are calculated*

Where "**W**" is the knapsack capacity.

Space Complexity:

Space Complexity	*Description*
O(nW)	*Uses a 2D DP table to store solutions for items and capacities.*

PRACTICAL – 6

Implementation of chain matrix multiplication using dynamic programming.

Aim:

To implement the **Matrix Chain Multiplication problem using Dynamic Programming** and determine the optimal order of multiplying a sequence of matrices so that the total number of scalar multiplications is minimized.

Objective:
The objective of this experiment is to understand and implement the **Matrix Chain Multiplication problem using Dynamic Programming**. It aims to find the most efficient order of multiplying a given sequence of matrices by minimizing the total number of scalar multiplications required. The experiment also helps in understanding the concepts of optimal substructure and overlapping subproblems in dynamic programming.

Algorithm:

- Start.
- Read the number of matrices n.
- Read the dimensions of the matrices in an array p.
  - Matrix A1 has dimensions p[0] × p[1].
  - Matrix A2 has dimensions p[1] × p[2].
  - Continue similarly for all matrices.
- Create a DP table m[n+1][n+1].
- Set m[i][i] = 0 because multiplying a single matrix requires no multiplication.
- Consider chain lengths from 2 to n.
- For every possible starting matrix i, calculate the ending matrix j.
- Try every possible split position k between i and j.
- Calculate the cost using:
  - cost = m[i][k] + m[k+1][j] + p[i-1] × p[k] × p[j]
- Store the minimum cost in m[i][j].
- The value m[1][n] gives the minimum number of scalar multiplications.
- Display the minimum multiplication cost.
- Stop.

Code:

```
# Matrix Chain Multiplication using Dynamic Programming

def matrix_chain_order(p, n):
    # m[i][j] stores minimum multiplication cost
    m = [[0 for j in range(n)] for i in range(n)]
```

```
    # Length of the chain
    for length in range(2, n):
        for i in range(1, n - length + 1):
            j = i + length - 1
            m[i][j] = float('inf')

            # Try all possible split positions
            for k in range(i, j):
                cost = (m[i][k] +
                        m[k + 1][j] +
                        p[i - 1] * p[k] * p[j])

                if cost < m[i][j]:
                    m[i][j] = cost

    return m[1][n - 1]


# Take input from the user
n = int(input("Enter the number of matrices: "))

print("Enter the dimensions:")

p = []

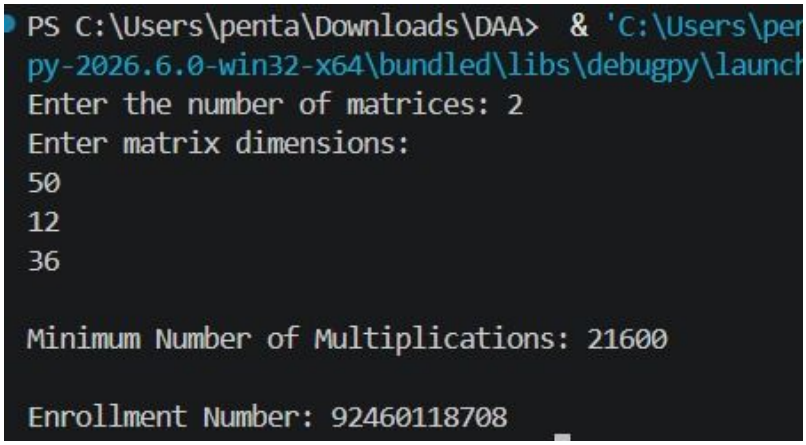
for i in range(n + 1):
    p.append(int(input()))

# Find minimum multiplication cost
minimum_cost = matrix_chain_order(p, n + 1)

# Display result
print("\nMinimum number of scalar multiplications:",
      minimum_cost)

# Enter your EN.No
print("EN.No:92460118708")
```

Output:



Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*$O(n^3)$*	*All possible matrix split positions are checked.*
Average Case	*$O(n^3)$*	*Evaluates different splits.*
Worst case	*$O(n^3)$*	*Checks all possible splits.*

Space Complexity:

Space Complexity	*Description*
$O(n^2)$	*A 2D table is used to store the minimum multiplication costs for different matrix chains.*

PRACTICAL – 7

Implementation of making a change problem using dynamic programming.

Aim:

To implement the **Making Change Problem using Dynamic Programming** and determine the minimum numberofcoinsrequiredtomakeagivenamount.

Objective:

The objective of this experiment is to solve the coin change problem efficiently using dynamic programming by breaking it into smaller subproblems. The program calculates the minimum number of coins required to make a given amount from the available denominations and stores intermediate results to avoid repeated calculations.

Algorithm:

- Start.
- Read the number of available coin denominations n.
- Read the value of each coin.
- Read the target amount amount.
- Create an array dp of size amount + 1.
- Initialize all elements of dp with a large value (INF).
- Set dp[0] = 0.
- For each amount i from 1 to amount:

  For every coin denomination:

  If the coin value is less than or equal to i, update:

  $dp[i]=\min(dp[i],dp[i-coin]+1)$
- If dp[amount] is still INF, print that change cannot be made.
- Otherwise, print the minimum number of coins required.
- Stop.

Code:

```
# Making Change Problem using Dynamic Programming

def making_change(coins, amount):
    # dp[i] stores the minimum number of coins
    # required to make amount i
    dp = [float('inf')] * (amount + 1)

    # 0 coins are required to make amount 0
    dp[0] = 0
```

```
    # Calculate minimum coins for each amount
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount]


# Take input from the user
n = int(input("Enter the number of coin denominations: "))

print("Enter the coin denominations:")
coins = []

for i in range(n):
    coins.append(int(input()))

amount = int(input("Enter the amount: "))

# Find minimum number of coins
minimum_coins = making_change(coins, amount)

# Display result
if minimum_coins == float('inf'):
    print("Change cannot be made for the given amount.")
else:
    print("\nMinimum number of coins required:", minimum_coins)

# Enter your EN.No
print("EN.No:92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\pen
py-2026.6.0-win32-x64\bundled\libs\debugpy\launch
Enter the number of coin denominations: 3
Enter the coin denominations:
2
3
4
Enter the amount: 6

Minimum number of coins required: 2
EN.No:92460118708
```

Time Complexity:

Case	Time Complexity	Description
Best Case	*O(n x A)*	Checks all n coins for each amount up to A.
Average Case	*O(n x A)*	The same DP table and coin checks are performed.
Worst case	*O(n x A)*	All amounts and all coin denominations are checked.

Space Complexity:

Space Complexity	Description
O(A)	A DP array of size A + 1 is used.

PRACTICAL – 8

Implementation of Graph and Searching (DFS and BFS)

Aim:

To implement graph traversal techniques **Depth First Search (DFS)** and **Breadth First Search (BFS)** and traverse all the vertices of a graph.

Objective:

The objective of this experiment is to understand and implement graph traversal techniques using DFS and BFS. DFS explores a graph by going as deep as possible before backtracking, while BFS explores the graph level by level using a queue.

Algorithm:

Depth First Search (DFS)

- Start.
- Read the number of vertices in the graph.
- Read the adjacency matrix of the graph.
- Read the starting vertex.
- Create a visited array and initialize all vertices as unvisited.
- Mark the starting vertex as visited.
- Display the starting vertex.
- For every adjacent vertex:
  If the vertex is not visited, recursively perform DFS on that vertex.
- Continue until all reachable vertices are visited.
- Stop.

Breadth First Search (BFS)

- Start.
- Read the number of vertices in the graph.
- Read the adjacency matrix of the graph.
- Read the starting vertex.
- Create a visited array and initialize all vertices as unvisited.
- Create an empty queue.
- Mark the starting vertex as visited and insert it into the queue.
- Remove a vertex from the queue and display it.

- Check all adjacent vertices.
- If an adjacent vertex is not visited, mark it as visited and insert it into the queue.
- Repeat until the queue becomes empty.
- Stop.

Code:

```
# Implementation of Graph Searching using DFS and BFS

# DFS function
def DFS(graph, vertex, visited):
  visited[vertex] = True
  print(vertex, end=" ")

  for i in range(len(graph)):
    if graph[vertex][i] == 1 and visited[i] == False:
      DFS(graph, i, visited)


# BFS function
def BFS(graph, start):
  visited = [False] * len(graph)
  queue = [start]
  visited[start] = True

  while queue:
    vertex = queue.pop(0)
    print(vertex, end=" ")

    for i in range(len(graph)):
      if graph[vertex][i] == 1 and visited[i] == False:
        visited[i] = True
        queue.append(i)


# Input
n = int(input("Enter the number of vertices: "))

print("Enter the adjacency matrix:")

graph = []
```

```
for i in range(n):
    row = list(map(int, input().split()))
    graph.append(row)

start = int(input("Enter the starting vertex: "))

# DFS
visited = [False] * n

print("\nDFS Traversal:")
DFS(graph, start, visited)

# BFS
print("\nBFS Traversal:")
BFS(graph, start)

print("\nEN.No:9246018708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\
py-2026.6.0-win32-x64\bundled\libs\debugpy\lau
Enter the number of vertices: 4
Enter the adjacency matrix:
0 1 2 1
0 1 0 0
1 2 0 1
0 0 1 0
Enter the starting vertex: 0

DFS Traversal:
0 1 3 2
BFS Traversal:
0 1 3 2
EN.No:92460118708
```

Time Complexity:

Case	Time Complexity	Description
Best Case	$O(V^2)$	Checks the adjacency matrix.
Average Case	$O(V^2)$	Checks vertices and connections.
Worst case	$O(V^2)$	Checks all vertices and connections

Space Complexity:

Space Complexity	Description
O(V)	Used for the visited array and recursion stack.

PRACTICAL – 9

Implementation of prim's algorithm.

Aim:
To implement **Prim's Algorithm** to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

Objective:
To implement **Prim's Algorithm** for finding the **Minimum Spanning Tree (MST)** of a connected, weighted,undirected graph by selecting the minimum-weight edge at each step without formingacycle.

Algorithm:

- Startwithanyvertex, usually vertex 0.
- Mark it as visited.
- Findtheminimum-weight edge connecting a visited vertex to an unvisited vertex.
- Addthatedgetothe MST.
- Markthenewlyselected vertex as visited.
- Repeatuntilallvertices are included in the MST.
- Calculatethetotalweight of the selected edges.

Code:

```
V = 5
graph = [
   [0, 2, 0, 6, 0],
   [2, 0, 3, 8, 5],
   [0, 3, 0, 0, 7],
   [6, 8, 0, 0, 9],
   [0, 5, 7, 9, 0]
]
visited = [False] * V
key = [float('inf')] * V
parent = [-1] * V

key[0] = 0

for count in range(V):
   minimum = float('inf')
```

```
u = -1

    for i in range(V):
        if not visited[i] and key[i] < minimum:
            minimum = key[i]
            u = i

    if u == -1:
        break

    visited[u] = True

    for v in range(V):
        if graph[u][v] != 0 and not visited[v] and graph[u][v] < key[v]:
            key[v] = graph[u][v]
            parent[v] = u

total = 0
print("Edge    Weight")

for i in range(1, V):
    print(parent[i], "-", i, "    ", graph[i][parent[i]])
    total += graph[i][parent[i]]

print("Total weight of MST =", total)
print("EN.No: 92460118708")
```

Output:



Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(V^2)*	*Checks vertices to find the minimum edge.*
Average Case	*O(V^2)*	*Selects minimum-weight edges.*
Worst case	*O(V^2)*	*Checks all vertices in each iteration.*

Space Complexity:

Space Complexity	*Description*
O(V^2)	**A 2D matrix is used to store the graph.**

PRACTICAL – 10

Implement Kruskal's algorithm

Aim:
To implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

Objective:
To implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph by selecting edges in increasing order of weight without forming cycles.

Algorithm:

- Sortall edges in ascending order of their weights.
- Select the edge with the minimum weight.
- Check whether adding the edge creates a cycle.
- If no cycle is formed, add the edge to the MST.
- Repeat until \(V-1\) edges are selected.
- Calculate and display the total weight of the MST.

Code:

```
V = 5

edges = [
    (0, 1, 2),
    (0, 3, 6),
    (1, 2, 3),
    (1, 3, 8),
    (1, 4, 5),
    (2, 4, 7),
    (3, 4, 9)
]

edges.sort(key=lambda x: x[2])

parent = list(range(V))
rank = [0] * V
```

```
def find_parent(i):
    if parent[i] != i:
        parent[i] = find_parent(parent[i])
    return parent[i]

def union_set(x, y):
    root_x = find_parent(x)
    root_y = find_parent(y)

    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y
    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x
    else:
        parent[root_y] = root_x
        rank[root_x] += 1

total = 0
count = 0

print("Edges in MST:")

for u, v, weight in edges:
    root_u = find_parent(u)
    root_v = find_parent(v)

    if root_u != root_v:
        print(u, "-", v, ":", weight)
        total += weight
        union_set(root_u, root_v)
        count += 1

    if count == V - 1:
        break

print("Total weight of MST =", total)
print("EN.No: 92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\penta\AppData\Local\Programs\Python\Python310\python.exe
py-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55069' '--' 'C:\Users\penta\Downloads\DAA\practi
Edges in MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total weight of MST = 16
EN.No: 92460118708
PS C:\Users\penta\Downloads\DAA>
```

Time Complexity:

Operation	*Time Complexity*	*Space Complexity*
Sorting edges	*O(E log E)*	**O(1) auxiliary**
Find and Union	*O(E alpha(V))*	**O(V)**
Overall	*O(E log E)*	**O(V) auxiliary**

PRACTICAL – 11

Write a program for Floyd-Warshal algorithm

Aim:
To write a C++ program to implement the Floyd–Warshall algorithm for finding the shortest paths between all pairs of vertices in a weighted graph.

Objective:
To calculatetheshortest distance between every pair of vertices using the dynamic programmingtechnique.

Algorithm:

1. Start.
2. Initializethe distance matrix using the given graph's edge weights.
3. Set thedistance from each vertex to itself as 0.
4. Considereach vertex k as an intermediate vertex.
5. For everypair of vertices i and j, update the distance using:
   [dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])]
6. Repeatuntil all vertices have been considered as intermediate vertices.
7. Displaythe shortest distance matrix.
8. Stop.

Code:

```
V = 4
INF = 99999

graph = [
   [0, 5, INF, 10],
   [INF, 0, 3, INF],
   [INF, INF, 0, 1],
   [INF, INF, INF, 0]
]

dist = [row[:] for row in graph]

for k in range(V):
   for i in range(V):
      for j in range(V):
```

```
            if (dist[i][k] != INF and
                dist[k][j] != INF and
                dist[i][k] + dist[k][j] < dist[i][j]):

                dist[i][j] = dist[i][k] + dist[k][j]

    print("Shortest Distance Matrix:")

    for i in range(V):
        for j in range(V):
            if dist[i][j] == INF:
                print("INF", end="\t")
            else:
                print(dist[i][j], end="\t")
        print()

    print("EN.No: 92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\penta\AppData\Local\Programs\Python\Python310\python.exe' 'c:\U
py-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52235' '--' 'C:\Users\penta\Downloads\DAA\practical 11\
Shortest Distance Matrix:
0       5       8       9
INF     0       3       4
INF     INF     0       1
INF     INF     INF     0
EN.No: 92460118708
PS C:\Users\penta\Downloads\DAA>
```

Time Complexity:

Space Comp	*Time Complexity*	*Operation*
O(V^2)	*O(V^2)*	**Initialize distance matrix**
O(1)auxiliary	*O(V^3)*	**Floyd–Warshall algorithm**
O(V^2)	*O(V^3)*	**Overall**

PRACTICAL – 12

Write a program for travelling salesman problem.

Aim:
To write a Python program to solve the Travelling Salesman Problem (TSP) and find the minimum cost tour that visits every city exactly once and returns to the starting city.

Objective:
To find the minimum-cost tour for a set of cities by checking all possible tours.

Algorithm:

1. Start.
2. Initialize the cost matrix containing the distances between cities.
3. Set the minimum cost to infinity.
4. Generate all possible routes starting from the first city.
5. Calculate the total cost of each route.
6. Add the cost of returning to the starting city.
7. If the total cost is less than the minimum cost, update the minimum cost and best route.
8. Repeat until all possible routes are checked.
9. Display the minimum cost and the best tour.
10. Stop.

Code:

```
from itertools import permutations

n = 4
INF = 99999

cost = [
   [0, 10, 15, 20],
   [10, 0, 35, 25],
   [15, 35, 0, 30],
   [20, 25, 30, 0]
]

cities = list(range(1, n))
min_cost = INF
best_path = []
```

```
for path in permutations(cities):
    total = cost[0][path[0]]

    for i in range(len(path) - 1):
        total += cost[path[i]][path[i + 1]]

    total += cost[path[-1]][0]

    if total < min_cost:
        min_cost = total
        best_path = [0] + list(path) + [0]

print("Minimum Cost:", min_cost)
print("Best Tour:", " -> ".join(str(city + 1) for city in best_path))
print("EN.No: 92460118708")
```

Output:



Time Complexity:

The program checks all possible tours. There are \((n-1)!\) possible tours, and calculating the cost of each tour takes O(n) time.
Overall time complexity: O(n!)

Space Comp	*Time Complexity*	*Operation*
O(n^2)	O(n^2)	Initialize cost matrix
O(n)	O((n-1)!)	Generate all possible tours
O(n)	O(n!)	Calculate tour cost and find minimum